

**MINISTRY OF EDUCATION AND RESEARCH**
**OF THE REPUBLIC OF MOLDOVA**
**Technical University of Moldova**
**Faculty of Computers, Informatics and Microelectronics**
**Department of Software Engineering and Automation**

**Moisei Daniel, FAF-241**
**Strulea Teodor, FAF-242**

# Report

*Laboratory Work No. 1*

***Network Programming (PR)***

Topic: Concurrency in Practice

CPU-bound and I/O-bound Work, Race Conditions, Speed-up

Checked by:
**Fiştic Cristofor**, *assistant lecturer*

**Chişinău – 2026**

## CONTENTS

**Introduction** **4**

**1 Short Theory** **5**
1.1 CPU-bound and I/O-bound Work . . . 5
1.2 Cores, Logical Processors and Speed-up . . . 5
1.3 Race Conditions . . . 5
1.4 Thread Pool . . . 5

**2 Environment and Measurement Method** **5**
2.1 Hardware and Software . . . 5
2.2 Input and Correctness . . . 6
2.3 Timing, Memory and Aggregation . . . 6

**3 Implementation** **8**
3.1 Program Structure . . . 8
3.2 Parsing Without Strings . . . 8
3.3 Task 1a: Dividing the File Without Splitting a Line . . . 9
3.4 Task 1b: Shared Counters . . . 10
3.5 Task 1c: Producer and Consumers . . . 10
3.6 Task 2: Files on the USB Drive . . . 11
3.7 Tools Used and Why . . . 12

**4 Task 1: Counting One File** **14**
4.1 Task 1a: Counting with Threads . . . 14
4.2 Experiment: Own Threads or Thread Pool . . . 15
4.3 Task 1b: Three Ways to Count . . . 15
4.4 Task 1c: Producer and Consumers . . . 16
4.5 Task 1c with the Thread Pool . . . 18

**5 Task 2: Many Files on Slow Storage** **19**

**6 Control Questions** **21**
6.1 Physical Core and Logical Processor . . . 21
6.2 No Line Counted Twice or Skipped . . . 21
6.3 Why Unsynchronised Shared Counters Lose Updates . . . 21
6.4 Why Counters of the Threads Are Faster Than a Lock . . . 21
6.5 I/O-bound for a USB Drive and for a Network Server . . . 21
6.6 Equal Parts in Advance or a Queue . . . 21

**7 Conclusions** **23**

**8 Source Code** **23**

## INTRODUCTION

The purpose of this laboratory work is to measure how the number of threads, and the way the work is divided among them, change the running time and the memory of a real workload: counting the negative, zero and positive integers of a text file with 50 000 000 numbers. From the measurements we decide when the work is CPU-bound and when it is I/O-bound, observe a race condition and the price of removing it, and find where the parallel speed-up stops.

The work was done jointly by Moisei Daniel (FAF-241) and Strulea Teodor (FAF-242). The program is written in C# (.NET 10) and follows the style of the lectures: threads are created with `new Thread`, every thread writes only into its own slot of an array, shared counters are protected by `lock (gate)`, and the producer and the consumers use a blocking queue. Every task is one class that implements the interface `ITask`, and the task and its settings are chosen on the command line. In addition to the tasks of the assignment, we compared our own threads with threads taken from the thread pool (Tasks 1a and 1c). Every run appends one row to a csv file (`task1a_results.csv`, …), and all tables, charts and numbers of this report are produced from these files by a script.

# 1 SHORT THEORY

## 1.1 CPU-bound and I/O-bound Work

A program is *CPU-bound* when its speed is limited by the processor: it keeps the processor busy, and more cores make it faster. It is *I/O-bound* when it spends most of its time waiting for input and output, for example for a file on a disk or a USB drive, or for the network; then the processor is mostly idle and the speed is limited by the device.

## 1.2 Cores, Logical Processors and Speed-up

A physical core is a complete computing unit. With hyper-threading the core has two "inboxes" and the operating system sees two logical processors; when one task waits (for data from memory, for example), the core switches to the other. In practice the gain is 15–30%, and a program that already keeps the core fully busy gains almost nothing. If $T(p)$ is the time with $p$ threads, the speed-up is

$$S(p) = \frac{T(1)}{T(p)}. \tag{1}$$

## 1.3 Race Conditions

The statement `counter++` is three operations: read, add, write. A thread can be stopped between any two of them, so two threads that increment the same variable can overwrite each other's result. A race needs shared mutable state and more than one thread; the fixes of the lecture remove one of them: do not share (a slot or a variable for each thread), or make the access exclusive with `lock`.

## 1.4 Thread Pool

With `new Thread` we own the thread and pay for creating it. With `Task.Run` or `ThreadPool.QueueUserWorkItem` we submit work and the runtime owns the threads: it keeps a limited number of them and reuses them. The rule of the lecture is never to block a pool thread, because while it waits it serves nobody. On this machine the pool starts with as many threads as logical processors (16) and adds more only slowly.

# 2 ENVIRONMENT AND MEASUREMENT METHOD

## 2.1 Hardware and Software

Table 1 describes the machine. The data was collected with `lscpu`, `free`, `lsblk` and `lsusb -t`; the raw output is saved in `data/hardware.txt`. The USB version was read from the kernel (`/sys/bus/usb/devices`): the drive declares USB 2.00 and is attached at 480 Mbit/s, although the laptop also has 10 Gbit/s ports.

Table 1: Hardware and software

| **Item** | **Value** |
|---|---|
| Processor | AMD Ryzen 7 6800HS, up to 4.79 GHz |
| Cores and threads | 8 physical cores, 16 logical processors (2 threads per core), L2 4 MiB, L3 16 MiB |
| Memory | 14 GiB RAM |
| Internal disk | Micron 2450 NVMe SSD, 954 GB |
| USB drive | “General USB Flash Disk” (ID `ffff:5678`), 29.3 GB, ext4; USB 2.0, 480 Mbit/s |
| Operating system | CachyOS, Linux 7.3.0-4-cachyos, x86-64 |
| Language and runtime | C# on .NET 10.0.13 (SDK 10.0.113), Release build |

### 2.2 Input and Correctness

`numbers.txt` contains 50 000 000 random integers in $[-1\,000\,000,\ 1\,000\,000]$, one per line, produced by `System.Random` with seed 12345; the file has 369 454 226 bytes (352.3 MiB). The command `generate` counts the numbers while it writes them, so the correct answer is known:

| | |
|---|---|
| Negative | 25 007 776 |
| Zero | 22 |
| Positive | 24 992 202 |
| Total | 50 000 000 |

For Task 2 the command `split` cuts the file, in order, into eight files of 6 250 000 numbers. Every task prints the three counters and checks that they add up to 50 000 000. In the tables of this report all runs found exactly the counters above; the only exception is the unsynchronised version of Task 1b, which is wrong by design.

### 2.3 Timing, Memory and Aggregation

Every measurement is a separate process, so the peak memory belongs to that run only. The time is taken with a `Stopwatch` and includes opening the file, creating the threads, counting, waiting for the threads and adding the partial results; it does not include the start of the runtime, and in Task 1b it does not include loading the array. The peak memory is the peak working set of the process (`PeakWorkingSet64`).

Tasks 1a and 1c were run 5 times for every number of threads (own threads and thread pool alternately, after two warm-up runs, so the file was in the page cache and the results do not show the speed of a cold disk). Task 1b was run in three separate processes, each with three runs of version 1 (9 runs of version 1 in total), and Task 2 three times for every number of threads. The tables give the median (for Task 2 the mean of three runs); the charts also show every run and the minimum–maximum range.

The laptop was also running a desktop session, so differences of a few per cent between neighbouring configurations are within the noise.

# 3 IMPLEMENTATION

## 3.1 Program Structure

Every task is a class that implements the interface `ITask` with the single method `Run(string[] args)`, and contains all its logic, including the creation of its threads. `Program.cs` only chooses the class from the first command-line argument. The files are: `GenerateTask`, `SplitTask`, `Task1a`, `Task1b`, `Task1c`, `Task2`, `Counts` (the counters and the parser), `Args` (reading `--threads 8`) and `Results` (peak memory and the csv files).

Listing 1: Choosing the task in Program.cs

```
ITask? task = args[0] switch
{
    "generate" => new GenerateTask(),
    "split" => new SplitTask(),
    "task1a" => new Task1a(),
    "task1b" => new Task1b(),
    "task1c" => new Task1c(),
    "task2" => new Task2(),
    _ => null
};
```

## 3.2 Parsing Without Strings

The numbers of Tasks 1a and 2 are parsed directly from the bytes of the file in one pass: the digits are accumulated into an integer, a minus sign is remembered, and at every newline the number is counted. No string is created. The method returns how many bytes it used; the unfinished last line of a buffer is carried to the next block.

Listing 2: The parser of Counts.cs

```
for (int i = 0; i < text.Length; i++)
{
    byte b = text[i];

    if (b >= '0' && b <= '9')
    {
        number = number * 10 + (b - '0');
        hasDigits = true;
    }
    else if (b == '-')
    {
        minus = true;
    }
    else if (b == '\n')
    {
        if (hasDigits) Add(minus ? -number : number);
        number = 0;
```

```
        minus = false;
        hasDigits = false;
        used = i + 1;
    }
}
```

### 3.3 Task 1a: Dividing the File Without Splitting a Line

Cutting the file at $size \cdot i/N$ would usually stop in the middle of a line. The start of part $i$ is therefore the first byte after the first newline found at or after the byte *before* the cut; if that byte is itself a newline, the line begins exactly at the cut and is not skipped. Each thread asks for its own start and for the start of the next part, which is its end. The same position always gives the same answer, so a line belongs to exactly one thread. Each thread reads its part in blocks of 1 MiB with positional reads (`RandomAccess.Read`) and never reads the whole file.

Listing 3: Start of a part (Task1a.cs)

```
long position = size * index / partCount - 1;
byte[] buffer = new byte[4096];

while (position < size)
{
    int read = RandomAccess.Read(file, buffer, position);
    if (read == 0) break;

    int newLine = Array.IndexOf(buffer, (byte)'\n', 0, read);
    if (newLine >= 0)
        return position + newLine + 1;

    position += read;
}
return size;
```

The threads are created as in the lectures; the option `--pool` replaces them with tasks of the thread pool. Both versions write into their own element of the array `counts`, which is summed after the threads have finished.

Listing 4: Own threads and thread pool in Task1a.cs

```
if (usePool)
{
    Task[] tasks = new Task[threadCount];
    for (int i = 0; i < threadCount; i++)
    {
        int index = i;
        tasks[i] = Task.Run(() => { counts[index] = CountPart(file, size, index, threadCount
    ); });
    }
    Task.WaitAll(tasks);
```

```
}
else
{
    Thread[] threads = new Thread[threadCount];
    for (int i = 0; i < threadCount; i++)
    {
        int index = i;
        threads[i] = new Thread(() => { counts[index] = CountPart(file, size, index,
    threadCount); });
        threads[i].Start();
    }
    foreach (Thread thread in threads)
        thread.Join();
}
```

### 3.4 Task 1b: Shared Counters

The numbers are first loaded into an `int[]` (not measured), then the array is cut into equal ranges, one for each thread. The three versions differ only in the loop. In the first the threads increment shared counters without any protection, in the second every update is inside `lock (gate)`, in the third every thread counts in its own variable and the results are added after `Join`.

Listing 5: Version 2, shared counters under a lock (Task1b.cs)

```
long[] shared = new long[3];
object gate = new object();
...
for (int k = from; k < to; k++)
{
    lock (gate)
    {
        if (numbers[k] < 0) shared[0]++;
        else if (numbers[k] == 0) shared[1]++;
        else shared[2]++;
    }
}
```

Listing 6: Version 3, a counter for each thread (Task1b.cs)

```
Counts mine = new Counts();   // only this thread uses it
for (int k = from; k < to; k++)
    mine.Add(numbers[k]);
counts[index] = mine;
```

### 3.5 Task 1c: Producer and Consumers

One reader thread reads the file line by line, collects 10 000 lines in a `List<string>` and puts it into a `BlockingCollection` with the capacity of 100 batches. `Add` waits while the queue

is full; `GetConsumingEnumerable` waits while it is empty and ends after the reader has called `CompleteAdding`. Every worker counts into its own slot of an array.

Listing 7: Reader and workers of Task1c.cs

```
string? line;
while ((line = reader.ReadLine()) != null)
{
    batch.Add(line);
    if (batch.Count == BatchSize)
    {
        queue.Add(batch);   // waits here while the queue is full
        batch = new List<string>(BatchSize);
    }
}

if (batch.Count > 0)
    queue.Add(batch);

queue.CompleteAdding();   // no more batches: the workers can stop
...
// waits while the queue is empty, ends after CompleteAdding when nothing is left
foreach (List<string> batch in queue.GetConsumingEnumerable())
{
    foreach (string line in batch)
        counts.Add(int.Parse(line));
}
```

### 3.6 Task 2: Files on the USB Drive

The eight files are divided in contiguous blocks among the threads (8, 4 or 2 files for a thread with 1, 2 or 4 threads). A thread reads a whole file with one operation into one byte buffer, parses the buffer, and uses the same buffer for its next file. The time of the reading and the time of the parsing are measured separately, which is what shows whether the work is I/O-bound.

Listing 8: Reading and parsing a file in Task2.cs

```
using (FileStream stream = File.OpenRead(files[k]))
{
    length = (int)stream.Length;
    stream.ReadExactly(buffer, 0, length);   // the whole file at once
}
long afterRead = Stopwatch.GetTimestamp();

counts[index].AddText(buffer.AsSpan(0, length), true);
long afterParse = Stopwatch.GetTimestamp();
```

### 3.7 Tools Used and Why

For every task we used the simplest tool that does what the assignment asks; the alternatives from the lectures that were not used are named, with the reason.

**All tasks.** `Thread` with `Start` and `Join`, as in lecture 1: the assignment asks for threads created by the program. `Join` is enough to wait, because every task has one phase, so a `Barrier` (which waits for all workers at the end of each phase) is not needed. Every thread writes only into its own element of an array (`Counts[]`): this is the first step of the ladder in lecture 2, *don't share*, so there is no race, and the array is summed after `Join`. The time is measured with `Stopwatch`, a monotonic clock (`DateTime.Now` can jump), and the peak memory is `PeakWorkingSet64`, the highest amount of RAM the process has used; each measurement is a new process, so earlier runs do not count.

**Task 1a.** The threads read with `RandomAccess.Read(handle, buffer, offset)`. It reads at a given position and does not use or change a common file position, so all threads can share one open file without a lock. With one `FileStream` the threads would share one position (`Seek` followed by `Read` from two threads is a race), and one stream for each thread would work but opens the file many times. A buffer of 1 MiB for each thread keeps the memory small: a thread never holds more than one block. The experiment with the thread pool uses `Task.Run`: we submit work and the runtime owns the threads.

**Task 1b.** `lock (gate)`, where `gate` is a private object used only for this purpose, as the lecture asks (step 4). `Interlocked` (step 3) is another correct way for one variable, but the assignment asks for exactly three versions: no synchronisation, a lock or mutex, and a counter for each thread (step 1). `Mutex` is a heavier operating-system object meant to be shared between processes; `lock` is the light in-process mutex of C#. The loading of the array (not measured) uses `File.ReadLines` and `int.Parse`, which is the simplest way and does not keep 50 million strings in memory at once, as `ReadAllLines` would.

**Task 1c.** `BlockingCollection` with a capacity of 100 batches gives, in one object, everything the assignment describes. `Add` waits while the queue is full (the reader waits); `GetConsuming Enumerable` waits while it is empty (the workers wait); `CompleteAdding` closes the queue, and the loops of the workers end by themselves when it is empty, so no special "end" item is needed. These are the *Add* and *Take* of the producer–consumer picture of lecture 3. The alternatives:

- `SemaphoreSlim` limits *how many* threads are inside, or counts free places, but it carries no data (lecture 2). A bounded queue would need two semaphores (free places and filled places), a lock around an ordinary queue, and a way to tell the workers that the file has ended. `BlockingCollection` does all of this with less room for mistakes. A semaphore is the right tool to allow, for example, at most three downloads at a time.

- `ConcurrentQueue` is thread-safe but never blocks and has no limit. The workers would have to wait for items by busy waiting (a whole core burns) or by sleeping in a loop (they wake up late), the two bad ways of waiting in lecture 3, and the queue could grow without limit.

- `Channel<T>` can also be bounded, but its reading and writing are made for `async`/`await`. Our reader and workers are threads that compute, and blocking on an async method with `.Result` breaks rule 1 of lecture 3; `async` makes waiting cheap and does not make computing faster.

- An own queue with `lock` and `Monitor.Wait`/`Pulse` is allowed by the assignment, but it is more code and it is easy to lose a wake-up.

A batch is a `List<string>` of 10 000 lines, so the lock of the queue is taken once for 10 000 lines. The reader uses `StreamReader.ReadLine`, because the assignment says that it reads line by line.

**Task 2.** A file is read with `File.OpenRead` and `ReadExactly`: one call that fills one `byte[]` with the whole file (not a list of lines), repeating the read internally if the system returns fewer bytes. `File.ReadAllBytes` would also read a file at once, but it allocates a new array of about 46 MB for every file; with one buffer for each thread, reused for its next file, the peak memory is about the number of threads times the size of a file (72, 116 and 205 MB in Table 5). `Stopwatch.GetTimestamp` is taken before and after the reading and the parsing, to separate the two times. The files are divided in advance, in blocks: 8 equal files divide evenly among 1, 2 and 4 threads, so a queue would only add a lock and give nothing (control question 6.6).

# 4 TASK 1: COUNTING ONE FILE

## 4.1 Task 1a: Counting with Threads

Figure 1 and Table 2 show the run with 1 to 64 threads; the blue line is the program with its own threads (the answer to the assignment), the orange line is the same program with tasks of the thread pool (Section 4.2).

Figure 1: Task 1a: execution time, speed-up and peak memory

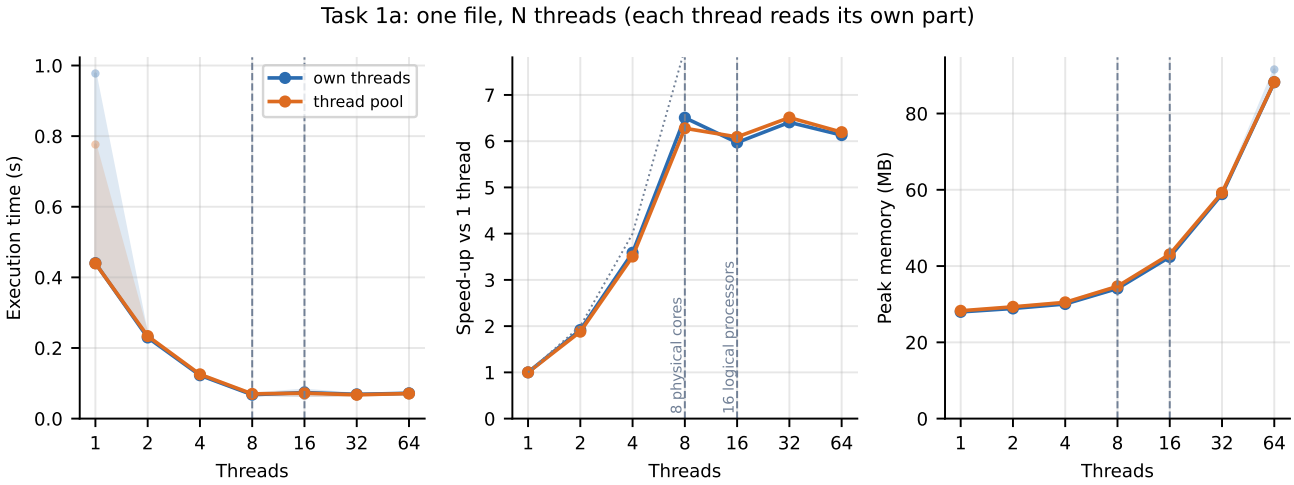



Table 2: Task 1a, median of 5 runs; the last three columns are the thread-pool variant

| Threads | Time (ms) | Speed-up | Peak mem. (MB) | Pool: time (ms) | Pool: speed-up | Pool: peak mem. (MB) |
|---|---|---|---|---|---|---|
| 1 | 440 | 1.00 | 28.0 | 440 | 1.00 | 28.3 |
| 2 | 230 | 1.92 | 28.9 | 234 | 1.88 | 29.3 |
| 4 | 123 | 3.59 | 30.1 | 125 | 3.51 | 30.5 |
| 8 | 68 | 6.51 | 34.1 | 70 | 6.28 | 34.7 |
| 16 | 74 | 5.97 | 42.4 | 72 | 6.09 | 43.1 |
| 32 | 69 | 6.41 | 58.9 | 68 | 6.51 | 59.2 |
| 64 | 72 | 6.13 | 88.2 | 71 | 6.20 | 88.2 |

**1. At which number of threads does the speed-up stop growing? Relate it to the physical cores and the logical processors.** The speed-up is close to ideal up to four threads (1.92 with two, 3.59 with four) and reaches 6.51 with eight threads, the number of physical cores: the time falls from 440 ms to 68 ms. After that it does not grow: with 16 threads, the number of logical processors, the speed-up is 5.97, and with 32 and 64 threads 6.41 and 6.13, all within the noise of the run-to-run variation. Hyper-threading therefore gave nothing here. This agrees with the lecture: the second logical processor only fills the moments in which a core waits, and our loop that parses the bytes keeps the core fully busy. That eight threads give 6.5 and not 8 times the speed is probably due to the memory and to the file cache that all threads share; we did not measure it.

**2. Why do 32 and 64 threads not help? Are they slower?** They are not slower: 69 ms and 72 ms against 68–74 ms for 8 and 16 threads. They do not help because the machine has only 16 logical processors: the additional threads wait for a time slice, and the work is already finished in about 70 ms, so there is nothing left to share.

**3. Does the peak memory depend on the number of threads? Why?** Yes. It grows from 28.0 MB with one thread to 88.2 MB with 64, about 1.0 MB for every thread. The file is never loaded as a whole: each thread has a read buffer of 1 MiB (and a stack), and the rest is the memory of the runtime.

### 4.2 Experiment: Own Threads or Thread Pool

To compare, the same program was run with tasks of the thread pool (`Task.Run`, option `--pool`) instead of threads created by us. In Task 1a the result is practically the same (Figure 1, Table 2): the times differ by at most 3% for every number of threads, and the peak memory is the same. The pool is **not faster** here, and the reason is that it has nothing to save. It reuses threads, which pays when there are many short tasks; here we need at most 64 threads for a job that lasts 70 to 440 ms, and creating a thread usually costs only tens of microseconds. With 32 and 64 parts the pool starts only 16 threads at once (its minimum is the number of logical processors; in a separate test, 64 tasks that each slept 300 ms ran at most 16 at a time and needed 1.2 s), and the other parts wait in its queue, but this does not change the time, because only 16 parts can run at once on this machine in any case. The pool has one more disadvantage for this assignment: we do not control the number of threads that run.

### 4.3 Task 1b: Three Ways to Count

The array of 50 000 000 integers was counted with 16 threads, the number of logical processors. Table 3 and Figure 2 show all runs.

Figure 2: Task 1b: time, peak memory and counted total of every run

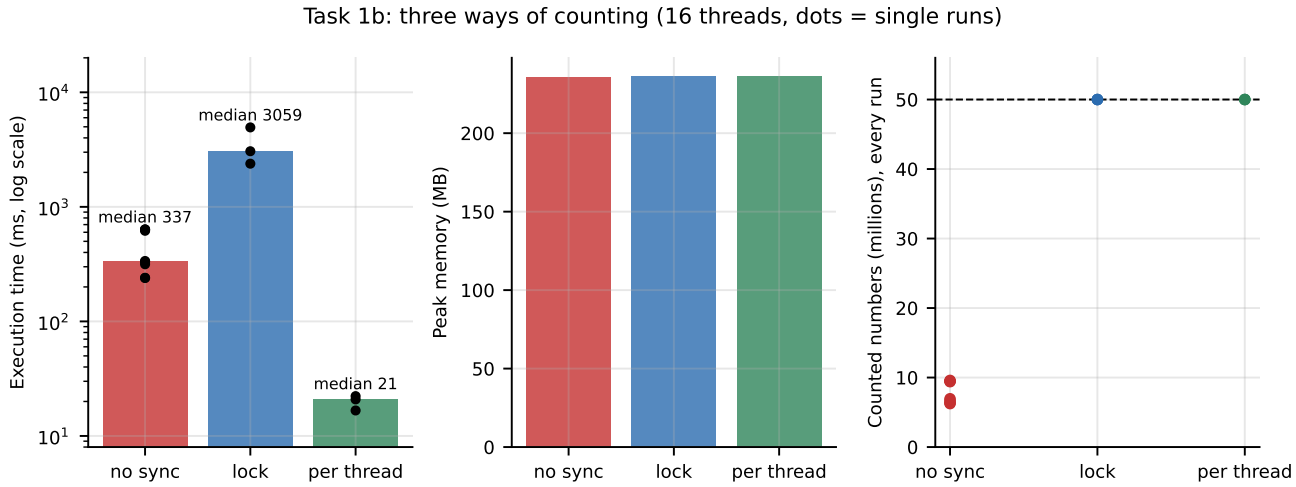


Table 3: Task 1b with 16 threads

| Version | Runs | Median (ms) | Min–max (ms) | Peak mem. (MB) | Counted total |
|---|---|---|---|---|---|
| 1. no synchronisation | 9 | 337 | 239–636 | 236 | 6 244 509–9 593 146 |
| 2. lock | 3 | 3059 | 2382–4939 | 236 | 50 000 000 |
| 3. counter per thread | 3 | 21 | 17–22 | 236 | 50 000 000 |

**4. Rank the three versions by time. Which are correct? Explain each result.** The fastest is version 3, the counters of the threads: 21 ms (17–22 ms in the three processes). Then comes version 1, without synchronisation: median 337 ms, 16 times slower than version 3, and its time changes much from run to run (239–636 ms). The slowest is version 2, the lock: median 3059 ms (2382–4939 ms), 147 times slower than version 3. Only versions 2 and 3 are correct. Version 1 counted only 6.2–9.6 million of the 50 million numbers (at most 19%); a short time of a wrong program has no value.

- *Version 1.* The three counters lie next to each other, in one cache line, and 16 threads write into it all the time, so the processors must pass the line to each other after almost every increment; this is why even this version is much slower than version 3, and why its time depends on where the counters happen to lie in memory. The read–add–write sequences of different threads overlap, so most of the updates are lost.

- *Version 2.* Every one of the 50 million updates takes the lock, so the counting is almost serial, and each acquisition also moves the lock and the counters between the processors and, under contention, puts the threads to sleep and wakes them.

- *Version 3.* Each thread counts in its own variable, there is nothing shared, and only three additions per thread are made at the end. The time is limited by the reading of the array.

The peak memory is 234–236 MB for all versions: it comes from the loading phase (the array alone has 190.7 MiB) and does not depend on the way of counting.

### 4.4 Task 1c: Producer and Consumers

Figure 3 and Table 4 show one reader and 1 to 64 workers, with a queue of 100 batches of 10 000 lines.

Figure 3: Task 1c: one reader, a variable number of workers

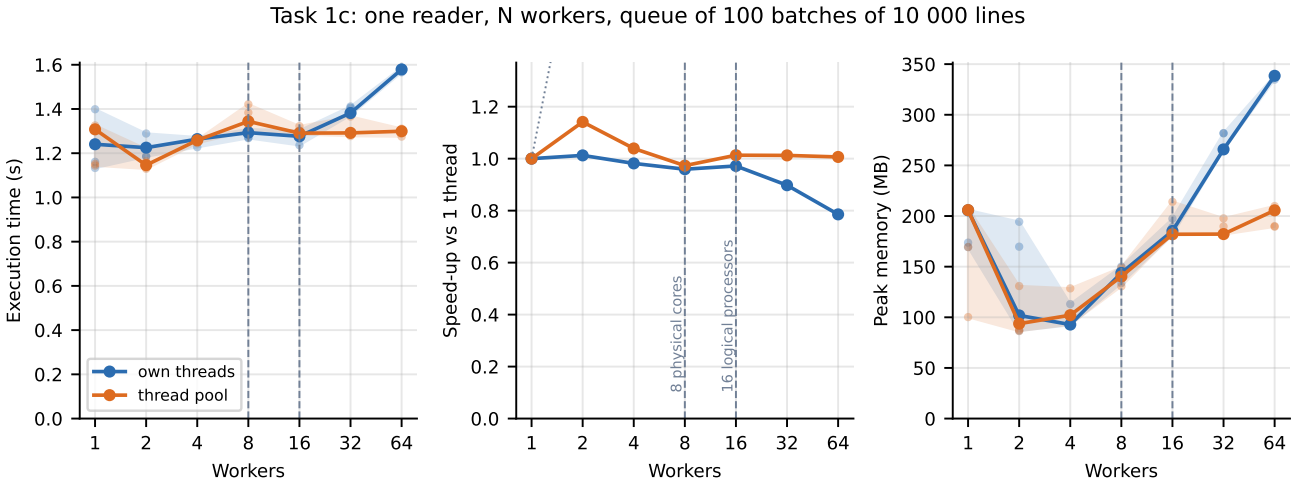



Table 4: Task 1c, median of 5 runs; the last three columns are the thread-pool variant

| Workers | Time (s) | Speed-up | Peak mem. (MB) | Pool: time (s) | Pool: speed-up | Pool: peak mem. (MB) |
|---|---|---|---|---|---|---|
| 1 | 1.241 | 1.00 | 205.7 | 1.308 | 1.00 | 205.8 |
| 2 | 1.225 | 1.01 | 101.9 | 1.146 | 1.14 | 93.7 |
| 4 | 1.264 | 0.98 | 92.8 | 1.259 | 1.04 | 102.1 |
| 8 | 1.293 | 0.96 | 143.8 | 1.344 | 0.97 | 140.3 |
| 16 | 1.276 | 0.97 | 185.2 | 1.291 | 1.01 | 182.0 |
| 32 | 1.382 | 0.90 | 265.8 | 1.292 | 1.01 | 182.2 |
| 64 | 1.579 | 0.79 | 338.5 | 1.299 | 1.01 | 205.6 |

**5. Compare Task 1c with Task 1a. Which is faster, and why? At which number of workers does Task 1c stop getting faster?** Task 1a is faster in every configuration: with one thread 0.440 s against 1.24 s for Task 1c (2.8 times more), and the best results are 0.068 s and 1.23 s (18 times). In Task 1a all threads read and parse in parallel. In Task 1c the single reader must read 50 million lines, create a string for each of them and put the batches into the queue, and this sequential work takes about 1.2 s, which is the time of the whole program; the workers only parse short strings and wait for the reader most of the time. Therefore Task 1c does not get faster with more workers at all: one worker is already as fast as the reader (1.24 s), the best result is with 2 workers (1.23 s, a speed-up of 1.01, which is within the variation of the runs with one worker, 1.13–1.40 s), and with more workers it becomes slower: 27% with 64 workers (1.58 s). The additional threads compete for the queue and cost memory (206 MB with one worker, 338 MB with 64). With one worker the memory is the highest of the small values, probably because the reader is then faster than the worker and the queue fills up; with two or more workers the queue stays almost empty. We did not measure the occupancy of the queue.

**6. What would happen with batches of a single line? And with a queue that has no size limit?** With one line in a batch the program would need 50 million operations on the queue instead of 5 000. Each of them takes a lock, may wake up a waiting thread and allocates a small list, so the cost of

the synchronisation would be much larger than the cost of parsing a number; the program would become much slower, and more workers would make it worse because they would fight for the same lock. (We did not run this configuration; the batch size of the assignment is fixed in the program.) With a queue without a limit the reader never has to wait. In our program the workers keep up with the reader, so the queue would stay short and nothing would change. If the reader were faster than the workers, the queue would keep growing until it contained the whole file as strings; as an estimate, 50 million strings of about 40 bytes plus a reference of 8 bytes each are about 2.4 GB, much more than the 352 MiB file. The limit makes the reader wait and keeps the memory small.

### 4.5 Task 1c with the Thread Pool

In Task 1c the reader and the workers were also started with `Task.Run`. Up to 16 workers the two variants have similar times (within about 7%, Table 4), but with 32 and 64 workers the pool is faster: 1.30 s against 1.58 s with 64 workers (18% less), and uses less memory (206 against 338 MB). This is not a real speed-up. The workers wait for the queue, which means they block pool threads, and the pool starts only 16 threads at once, so with 64 workers only about 16 can be active at a time, as in the separate test of Section 4.2 (the others wait for a free thread and are not needed), whereas our 64 threads all fight for the queue and all hold memory. The pool therefore works here as a limit of the number of workers: the best result of the pool (1.15 s with two workers) is not significantly better than the best result of our own threads (1.23 s). The lecture rule “never block a pool thread” still applies: this program works only because the reader is started before the workers and the workers are not needed in the end; a pool full of waiting workers would delay a task that is started later.

## 5 TASK 2: MANY FILES ON SLOW STORAGE

The eight files were copied to the USB drive (USB 2.0, Table 1). The drive was ejected, unplugged and plugged in again before every run, so no data came from the page cache. Each configuration (1, 2 and 4 threads) was run three times, interleaved in the order 1, 2, 4, 1, 2, 4, 1, 2, 4; Figure 4 and Table 5 give the mean of the three runs. For comparison the same program was run on the internal SSD (the files had been read recently, so they were probably in the page cache; the result shows the cost of parsing and not of the disk).

Figure 4: Task 2 on the USB drive: execution time, speed-up and peak memory

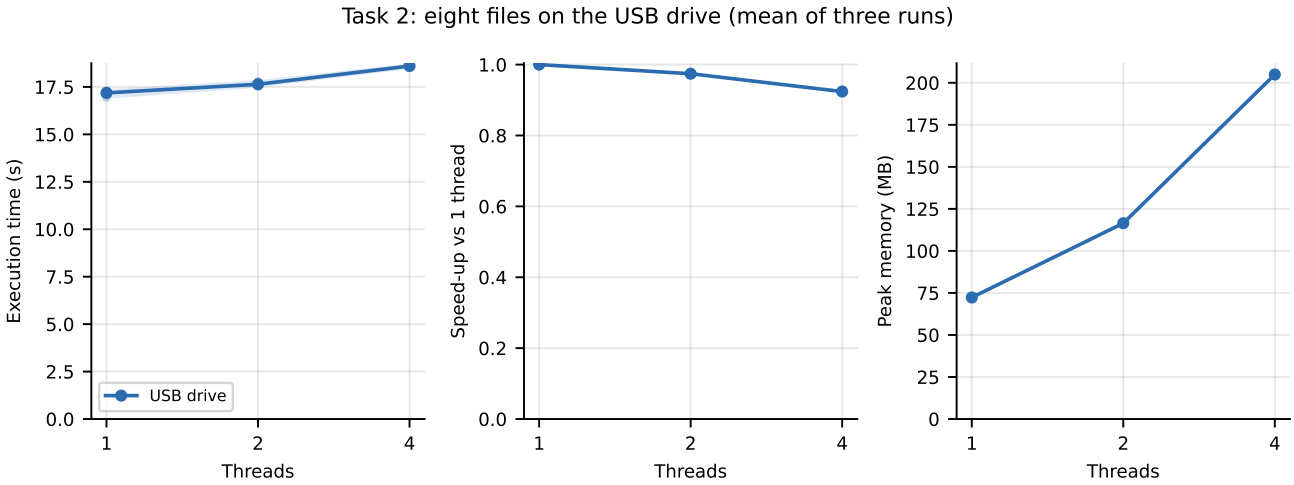
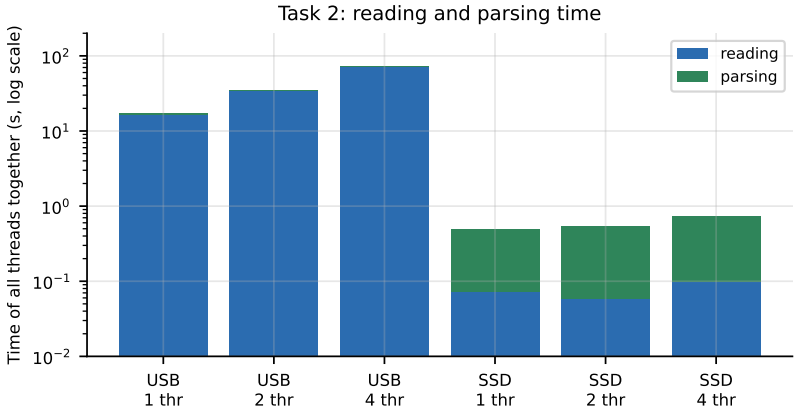



Table 5: Task 2: mean of three runs; reading and parsing are the times of all threads together

| Storage | Threads | Time (s) | Speed-up | Reading (s) | Parsing (s) | Peak mem. (MB) | Runs |
|---|---|---|---|---|---|---|---|
| USB drive | 1 | 17.19 | 1.00 | 16.43 | 0.75 | 72 | 3 |
| USB drive | 2 | 17.64 | 0.97 | 34.47 | 0.75 | 116 | 3 |
| USB drive | 4 | 18.60 | 0.92 | 71.88 | 0.74 | 205 | 3 |
| Internal SSD | 1 | 0.50 | 1.00 | 0.07 | 0.42 | 71 | 3 |
| Internal SSD | 2 | 0.28 | 1.78 | 0.06 | 0.49 | 115 | 3 |
| Internal SSD | 4 | 0.21 | 2.37 | 0.10 | 0.64 | 203 | 3 |

**1. Compare one thread with Task 1a on one thread. Is the work CPU-bound or I/O-bound?** On the USB drive one thread needs 17.19 s, against 0.440 s for Task 1a (39 times more) and 497 ms for the same program on the internal SSD (35 times less). The parsing is a small part of the time in both cases (0.75 s on the USB drive, 0.42 s on the SSD), but the reading takes 16.4 s on the USB drive and only 0.07 s on the SSD (Figure 5). On the USB drive 96% of the time is reading: a read speed of about 21 MiB/s, normal for a USB 2.0 flash drive. The work is therefore **I/O-bound** on the USB drive. On the SSD, in Task 1a and in all other tasks it is CPU-bound, with only 15% of the time in reading.

Figure 5: Task 2: reading and parsing time (log scale)




**2. Do more threads help on the USB drive? Why not, when lecture 1 showed that I/O-bound work benefits from concurrency?** No. The time is 17.19 s with one thread, 17.64 s with two (speed-up 0.97) and 18.60 s with four (0.92); the three runs of one configuration differ by less than 0.5 s, so the line is flat and, if anything, a little worse with more threads. The reading time of all threads together doubles and quadruples (16.4, 34.5 and 71.9 s), because the threads wait for each other: all of them read through one USB 2.0 link from one flash drive, so each of them receives a smaller share of the same bandwidth. A separate test confirms this (`dd` with direct I/O, which bypasses the cache, three repetitions, `data/usb-throughput.txt`): one reader gets 23.2 MiB/s, two readers 21–22 MiB/s together and four readers 20.6 MiB/s together. Concurrency helps an I/O-bound program when the waits are independent and the device has free capacity, as with a server that waits for many different clients, each answering in its own time; here there is one shared device that one thread already uses completely. On the SSD, where the work is mostly parsing, more threads do help (1.78 with two and 2.37 with four threads). More threads cost memory in Task 2: each thread holds a buffer for a whole file of about 46 MB, so the peak is 72, 116 and 205 MB.

# 6 CONTROL QUESTIONS

## 6.1 Physical Core and Logical Processor

A physical core is a complete computing unit; a logical processor is a hardware thread that the operating system can schedule, and with hyper-threading one core shows two of them that share its execution units. This processor has 8 cores and 16 logical processors. In our charts it is the place of the dashed lines in Figures 1 and 3: in Task 1a the speed-up grows almost linearly up to 8 threads (6.51) and does not grow between 8 and 16 threads (5.97) and further.

## 6.2 No Line Counted Twice or Skipped

The rough cut at $\mathit{size} \cdot i/N$ is moved to the first byte after the next newline, searching from the byte before the cut, so a line that starts exactly at the cut is not lost. The range of a thread is from its start to the start of the next thread, and both are computed by the same function, so each line belongs to exactly one thread. All runs of Task 1a, with every number of threads from 1 to 64, gave exactly the counters of the generator.

## 6.3 Why Unsynchronised Shared Counters Lose Updates

Suppose the counter holds 10 and two threads add one. (1) Thread A reads 10. (2) Thread B reads 10. (3) A computes 11. (4) B computes 11. (5) A writes 11. (6) B writes 11. Two increments were made, but the counter grew by one. With 16 threads this happens all the time: in our runs only 6.2–9.6 million of the 50 million updates remained.

## 6.4 Why Counters of the Threads Are Faster Than a Lock

A lock makes the 50 million updates run one after another, and every acquisition costs time even when nobody waits, because the lock and the counters move between the caches of the processors; under contention the threads are also put to sleep and woken. With a counter for each thread nothing is shared while counting, and the synchronisation is reduced from one per number to one per thread: 21 ms against 3059 ms in our runs.

## 6.5 I/O-bound for a USB Drive and for a Network Server

For a USB drive “I/O-bound” means that the time is spent waiting until the device delivers the data: in Task 2 96% of the time was reading at about 21 MiB/s. For a server waiting for the network the time is spent waiting for the other side. Concurrency helps in the network case because the waits belong to different, independent clients: while one request waits for its answer, the processor serves another, and the throughput grows. The USB drive is one device with one limited bandwidth, which one thread already uses completely, so more threads only wait for the same resource, as Task 2 showed.

## 6.6 Equal Parts in Advance or a Queue

Dividing the work in advance (Task 1a) costs nothing while the program runs: there is no shared structure and no lock, and every thread reads its own contiguous part. It is better when the parts need about the same time, as here. A queue (Task 1c) gives the next piece to the worker that is free, so it balances

parts of unequal cost, and it allows one producer to feed the workers when the data cannot be divided in advance (a stream, a network). The price is the lock of the queue, the batches in memory and a sequential producer, which in our case decided the time of the whole program: 1.23 s against 0.068 s for the division in advance.

## 7 CONCLUSIONS

The measurements on this machine (8 physical cores, 16 logical processors) show that the structure of the work matters more than the number of threads. Counting one file in independent, line-aligned parts (Task 1a) is CPU-bound: the speed-up is almost linear up to four threads, reaches 6.51 with the 8 physical cores and does not grow with 16, 32 and 64 threads, while the memory grows by about 1 MiB for every thread. Threads from the thread pool gave the same times as our own threads, because the work is long compared with the cost of creating a thread.

On the shared array the two correct versions differ much: the counters of the threads need 21 ms and the lock 3059 ms. Without synchronisation the program loses more than 80% of the updates and the result is wrong; its time varies from 239 to 636 ms because the threads share a cache line. The producer–consumer version (Task 1c) was 18 times slower than the independent parts, because one reader reads and builds the strings of 50 million lines; more workers did not help, and with 64 own threads the program became 27% slower. The thread pool avoided this only because it runs at most 16 threads at once.

Task 2 was I/O-bound: with the files on a USB 2.0 flash drive 96% of the time was spent reading at about 21 MiB/s, the run took 39 times longer than Task 1a, and extra threads gave no speed-up (0.97 and 0.92) because all of them waited for the same drive. In short, on this machine Tasks 1a, 1b and Task 2 on the SSD were CPU-bound, Task 1c was limited by its single reader, and Task 2 on the USB drive was I/O-bound. The results are limited by 5 runs for Tasks 1a and 1c, three runs for Task 2, and by the other programs that ran on the laptop; the conclusions concern differences that are large compared with this variation.

## 8 SOURCE CODE

The C# source code, the csv files with all measurements and this report are available in the repository:

`https://github.com/Moisei-D/PR-Labs`

The program is in the folder `Lab1` (one file for every task) and the report in `Reports/Lab1 Report`. The `README.md` of the project lists all commands; for example:

```
cd Lab1
dotnet build -c Release
dotnet run -c Release -- generate
dotnet run -c Release -- split
dotnet run -c Release -- task1a --threads 8
dotnet run -c Release -- task1a --threads 8 --pool
dotnet run -c Release -- task1b
dotnet run -c Release -- task1c --threads 8
dotnet run -c Release -- task2 --dir /path/to/usb --threads 2 --storage usb
```